

MODULE 2 · WEEK 3

Explainable AI (xAI)

Open the black box: partial dependence plots, permutation importance, and using them to tell a story and select features.

Lectures 11 · 14–15 · 18 · 24



This week

- 1 **Why xAI** — black-box models still need to be explained
- 2 **Partial dependence plots** — how a feature moves the prediction
- 3 **Linear vs. nonlinear** — what the shape of a PDP tells you
- 4 **Permutation importance** — which features actually matter
- 5 **autoML with TPOT** — let a search find the pipeline



Why explainability

- Powerful models (forests, boosting, nets) are accurate but opaque — data in, prediction out.
- Stakeholders need to know **which features drive predictions** and **which direction**.
- xAI turns a black box into something you can defend in a meeting and trust in production.

Partial dependence plots (PDPs)

- A PDP shows how the prediction changes as you sweep **one feature**, averaging over the rest.
- Reads like "as income goes up, predicted price goes up."
- It captures the model's learned relationship — not just a raw correlation.

```
from sklearn.inspection import PartialDependenceDisplay
PartialDependenceDisplay.from_estimator(
    model, X, features=['income', 'rooms'])
```

Linear vs. nonlinear responses

- A straight PDP = a roughly linear effect; a curve or step = a nonlinear effect the model found.
- Nonlinear shapes are exactly why we use flexible models — a linear regression would miss them.
- Early "clunky" PDPs are still useful: they reveal where the model bends.

Key idea The shape of a PDP tells you whether a relationship is straight, curved, or stepwise — insight a single coefficient can't give.

Permutation importance

- Shuffle one feature's values and see how much the score drops — a big drop means a big deal.
- Model-agnostic: works on any fitted model, measured on the **test** set.
- Run it several times and average — shuffling is random, so you get a distribution.

```
from sklearn.inspection import permutation_importance
r = permutation_importance(model, X_test, y_test,
                           n_repeats=10, random_state=42)
r.importances_mean
```

Key idea If shuffling a column barely changes the score, that column isn't earning its place.



xAI for storytelling & feature selection

- Use importance to **explain** the model — "rooms and income drive most of the prediction."
- Use it to **simplify** — drop low-importance features, refit, and check the score holds.
- A leaner model that scores the same is easier to explain, cheaper to run, and less likely to overfit.

autoML with TPOT

- **TPOT** uses a genetic search to assemble preprocessing + model pipelines automatically.
- It fits and mutates many candidate pipelines, keeping the best performers each generation.
- Great as a strong baseline — then you interpret and refine the winner yourself.

```
from tpot import TPOTClassifier
tpot = TPOTClassifier(generations=5, population_size=20,
                      cv=5, random_state=42)
tpot.fit(X_train, y_train)
```




Lab this week

- Fit a model, then explain it: build partial dependence plots and run permutation importance.
- Use the importance ranking to drop weak features, refit, and confirm performance holds — turning explanation into a leaner model.



Key takeaways

- Accurate models still need to be explained — that's what xAI is for.
- PDPs show how and in which direction a feature moves the prediction.
- Permutation importance ranks features by how much shuffling them hurts the score.
- Use xAI to tell the story and to trim features down to what matters.
- TPOT/autoML searches pipelines for you — a strong baseline to interpret, not a black box to trust blindly.